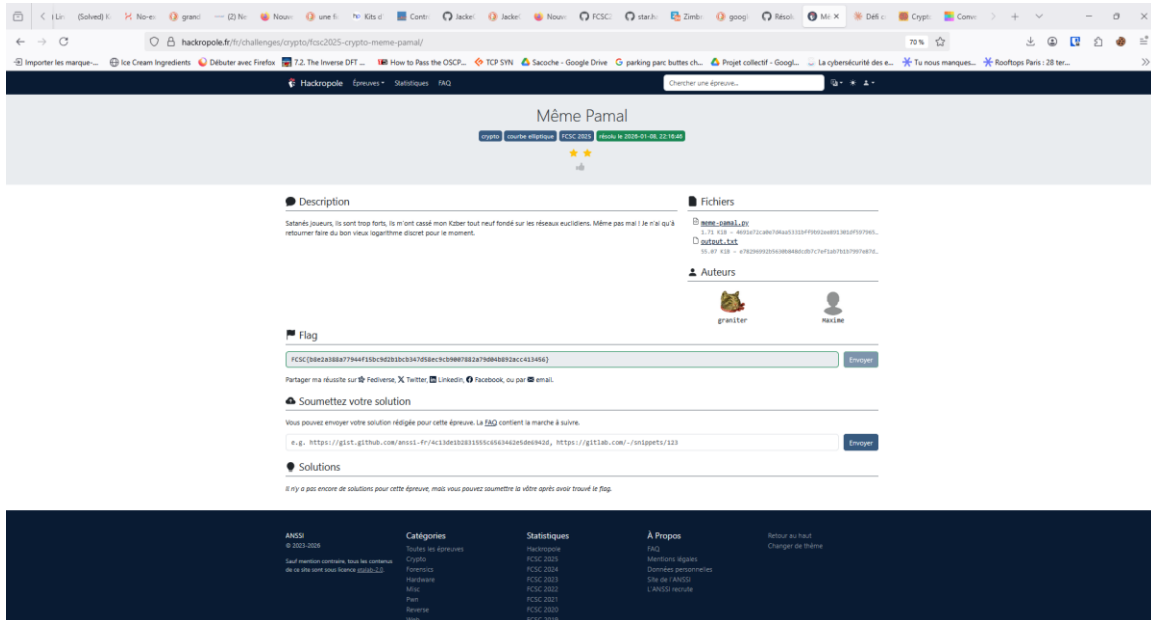


Même Pamal — FCSC 2025 (Crypto / Courbe elliptique)



Objectif

Objectif : retrouver le message (flag) chiffré dans output.txt / output(20).txt sans résoudre de logarithme discret, en exploitant une erreur de conception entre le sous-groupe utilisé pour ElGamal et l'encodage des octets via lift_x.

Mécanisme du chiffrement

Principe du chiffrement

Le challenge implémente un ElGamal additif sur courbe elliptique, appliqué octet par octet :

- 1) Encodage d'un octet m en point $t = \text{encode}(m)$ via lift_x (recherche d'un x valide).
- 2) Tirage d'un aléa r , puis $u = g \cdot r$.
- 3) Masquage : $v = t + \text{pk} \cdot r$, où $\text{pk} = g \cdot \text{sk}$ est la clé publique.

Le chiffré d'un octet est donc le couple (u, v) .

Pourquoi ça casse (idée centrale)

Le chiffrement est un ElGamal additif sur la courbe, mais dans un sous-groupe premier de taille order (généré par g , et donc $\text{pk} \cdot r \in \langle g \rangle$). En revanche, l'encodage $\text{encode}()$ fait un lift_x et produit un point t dans le groupe complet (pas forcé dans le sous-groupe).

Pour chaque octet : $v = t + \text{pk} \cdot r$. En multipliant par order , on annule la partie sous-groupe :

$$[\text{order}] \cdot v = [\text{order}] \cdot t + [\text{order}] \cdot (\text{pk} \cdot r) = [\text{order}] \cdot t$$

Donc on pré-calcule une table [order]*encode(b) pour b ∈ [0..255], puis on décode chaque v par lookup.

Fichiers fournis

- meme-pamal.py : script du challenge (paramètres de la courbe et algorithme).
- output.txt / output(20).txt : sortie JSON contenant la clé publique et le chiffré (liste de points).
- solve_meme_pamal.py : solveur (optionnel, si déjà fourni dans votre dossier).

Étapes réalisées

1. Lire les paramètres de la courbe et du sous-groupe (p, a, b, g, order) et la clé publique pk depuis meme-pamal.py / output.txt.
2. Identifier la forme du chiffré : pour chaque octet, on a (u, v) avec $u = g \cdot r$ et $v = \text{encode}(m) + pk \cdot r$.
3. Noter que $pk \cdot r$ est toujours dans le sous-groupe (g) d'ordre order (car $pk = g \cdot sk$).
4. Constater que l'encodage $\text{encode}(m)$ (via `lift_x`) ne contient pas t à appartenir à (g).
5. Multiplier chaque v par `order` : $V = [\text{order}] \cdot v = [\text{order}] \cdot \text{encode}(m)$ (la partie $pk \cdot r$ disparaît).
6. Pré-calculer une table pour tous les octets $b \in [0..255]$: $Tb = [\text{order}] \cdot \text{encode}(b) \rightarrow b$.
7. Pour chaque chiffré, calculer $V = [\text{order}] \cdot v$ et retrouver l'octet m via lookup dans la table.
8. Concaténer les octets pour reconstruire le message clair.
9. Vérifier le format attendu du flag (FCSC{...}).
10. Automatiser l'ensemble dans un solveur (option -v pour logs/diagnostics).

Exécution du solveur

Options attendues (voir `-h`) :

- --chal : chemin vers meme-pamal.py
- --out : chemin vers output.txt / output(20).txt (JSON)
- -v / --verbose : verbosité

Exemples :

```
python3 solve_meme_pamal.py --chal meme-pamal.py --out output.txt
python3 solve_meme_pamal.py --chal meme-pamal.py --out output.txt -v
```

```
kali@kali:~/Share/FCSC2025_keep/crypto/Meme_Pamal$ python3 solve_meme_pamal.py --chal meme-pamal.py --out output.txt
FCSC{2b38847f5bcb2d1b3d47d58ec9d08782a79d8b892ac413456}
kali@kali:~/Share/FCSC2025_keep/crypto/Meme_Pamal$
```

Remarques techniques

L'attaque ne nécessite aucun logarithme discret : elle repose sur l'annulation de la composante dans $\langle g \rangle$ par multiplication par order, puis sur un dictionnaire de 256 valeurs.

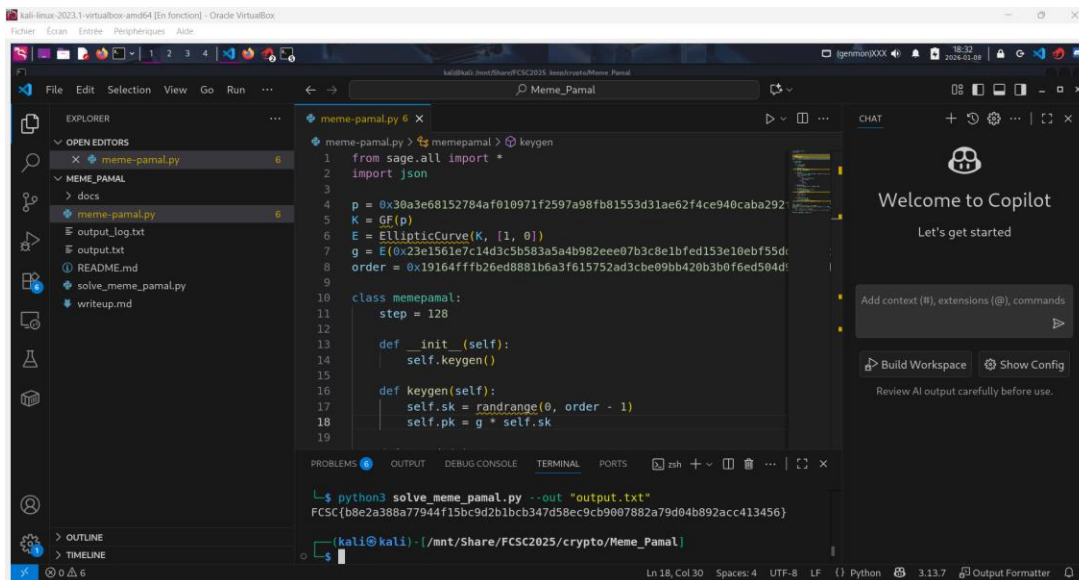
Dans une implémentation robuste, on éviterait d'encoder des messages en points hors sous-groupe (ou on ferait des checks de sous-groupe / cofacteur)¹ et on utiliserait plutôt ECIES (ECDH + KDF + AEAD).

Remarque : sur la pratique standard : **ECIES** :

- ECDH pour obtenir un secret partagé,
- KDF pour dériver une clé symétrique,
- AEAD pour chiffrer et authentifier le message

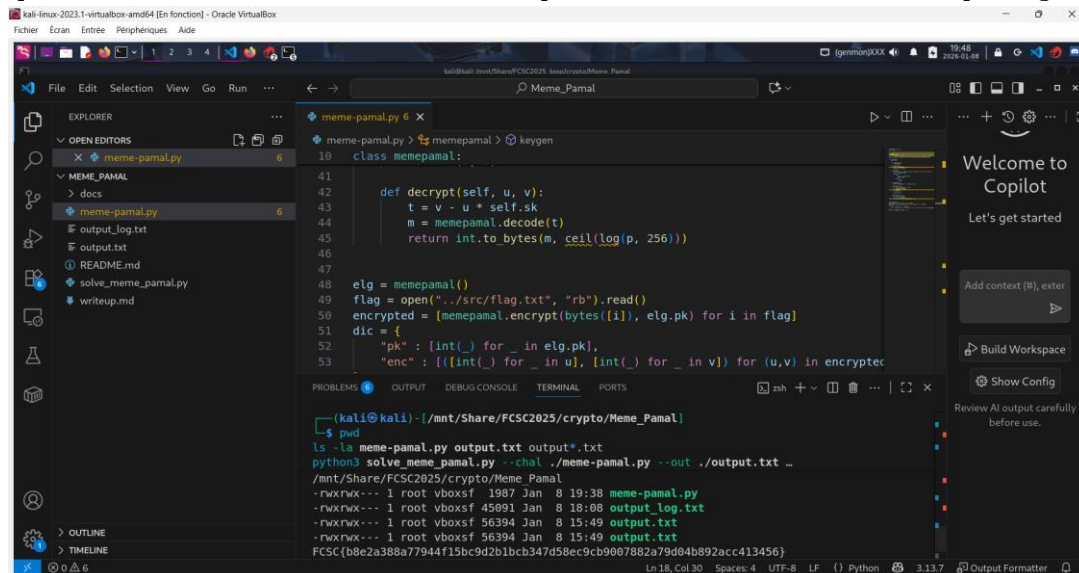
L'avantage : on ne mélange pas "message en point EC" et "masquage par addition" de façon fragile ; on reste dans une construction standardisée, audité, et moins sujette aux attaques de sous-groupe

🚩 **Flag FCSC{b8e2a388a77944f15bc9d2b1bcb347d58ec9cb9007882a79d04b892acc413456}**



```
1 from sage.all import *
2 import json
3
4 p = 0x30a3e68152784af010971f2597a98fb81553d31ae62f4ce940caba292
5 K = GF(p)
6 E = EllipticCurve(K, [1, 0])
7 g = E(0x23e1561e7c14d3c5b583a5a4b982eee07b3c8e1bfed153e10ebf55d
8 order = 0x19164ffffb26ed8881b6a3f615752ad3cbe09bb420b3b0f6ed504d
9
10 class memepamal:
11     step = 128
12
13     def __init__(self):
14         self.keygen()
15
16     def keygen(self):
17         self.sk = randrange(0, order - 1)
18         self.pk = g * self.sk
19
20
21 $ python3 solve_meme_pamal.py --out "output.txt"
22 FCSC{b8e2a388a77944f15bc9d2b1bcb347d58ec9cb9007882a79d04b892acc413456}
```

Capture de de l'écran de lancement du script conduisant au résultat :le drapeau (flag)



```
41 class memepamal:
42
43     def decrypt(self, u, v):
44         t = v - u * self.sk
45         m = memepamal.decode(t)
46         return int.to_bytes(m, ceil(log(p, 256)))
47
48 elg = memepamal()
49 flag = open("../src/flag.txt", "rb").read()
50 encrypted = [memepamal.encrypt(bytes[i], elg.pk) for i in flag]
51 dic = {
52     "pk" : [int(_) for _ in elg.pk],
53     "enc" : [(int(_) for _ in u), (int(_) for _ in v) for (u,v) in encrypted]
54 }
55
56 $ python3 solve_meme_pamal.py --chal ./meme-pamal.py --out ./output.txt
57 FCSC{b8e2a388a77944f15bc9d2b1bcb347d58ec9cb9007882a79d04b892acc413456}
```

¹ Pour éviter ce type de fuite structurelle : une operation est de forcer les points à vivre dans le bon sous-groupe :

- soit en encodant directement dans $\langle g \rangle$,
- soit en "cofactor clearing" : remplacer t par $[h] \cdot t$ (où h est le cofacteur),
- et/ou en faisant des **subgroup checks** (vérifier que $[order] \cdot P = \mathcal{O}$ quand on attend un point dans $\langle g \rangle$).